Informe ESP32

Introducción y objetivo

El proyecto es un detector de obstáculos estático: un ESP32 hace un barrido de 0° a 180°, o al revés, con un sensor de ultrasonidos montado sobre un servo, guarda en la EEPROM la distancia medida en cada ángulo y la publica por WiFi en una página web.

El robot construye un mapa de distancias por ángulo que se conserva aunque el equipo se apague o entre en sueño profundo. Cualquier ordenador o móvil de la misma red puede consultar ese mapa desde el navegador.

Requisitos

Requisito	Cómo se cumple
1. Sensor, actuador	Un sensor de ultrasonidos mide distancias y un servo lo hace girar para barrer el entorno
2. Comunicación por WiFi	El ESP32 funciona como servidor web y muestra los datos en el navegador
3. Modo de energía y despertar	Duerme en Deep Sleep y se despierta con un temporizador
4. EEPROM	Guarda el mapa de distancias para que no se pierda mientras duerme

Funcionamiento del ciclo

El ESP32 repite siempre el mismo ciclo: despierta, mide, guarda, publica y vuelve a dormir.

1. **Despertar.** Un temporizador enciende el ESP32 tras el periodo de reposo.
2. **Barrido.** El servo gira el sensor y se mide la distancia en varios ángulos.
3. **Guardado.** Las distancias se guardan en la EEPROM, que conserva los datos aunque el equipo duerma.
4. **Publicación.** El ESP32 se conecta al WiFi y muestra una tabla de ángulos y distancias en una página web durante un tiempo limitado. Los obstáculos más cercanos (<=20cm) se resaltan en rojo.
5. **Sueño.** Se desconecta y entra en Deep Sleep para ahorrar energía hasta el siguiente despertar.

Las cinco etapas se repiten en cada despertar; lo que pasa de un ciclo al siguiente está guardado en la EEPROM.

Explicación del código: sensor y barrido

El programa se organiza en funciones pequeñas, cada una con una tarea. `setup()` prepara el hardware y `loop()` ejecuta el ciclo completo: barrer, guardar, publicar y dormir.

Medir la distancia. `readDistance()` envía un pulso corto por el pin TRIG y mide con `pulseIn` cuánto tarda el eco en volver. El tiempo se convierte a centímetros dividiendo entre 29,2 (el sonido tarda unos 29,2 µs por cm) y entre 2 (el sonido hace ida y vuelta).

Barrer y guardar. `medirYGuardar(ang)` mueve el servo al ángulo indicado, espera 500 ms a que llegue, mide y guarda el valor redondeado a centímetros enteros en la EEPROM. `loop()` la llama para cada ángulo, de 0° a 180° o al revés según el último sentido guardado.

Explicación del código: EEPROM

Tras un Deep Sleep el ESP32 se reinicia y las variables normales se pierden, así que lo que se debe guardar se guarda en la EEPROM. Se reservan 20 bytes con `EEPROM.begin(20)`: uno para el último ángulo del servo y 19 para el mapa (una distancia por cada 10°).

Al arrancar se lee el último ángulo guardado y se barre en el sentido contrario, de modo que el servo no tiene que volver de golpe a la posición inicial. Los datos se escriben con `EEPROM.write()` y se graban de forma definitiva en la flash con `EEPROM.commit()`.

Explicación del código: comunicación por WiFi

La comunicación sigue el ejemplo visto en clase: el ESP32 se comporta como un servidor web y el navegador de cualquier dispositivo de la misma red actúa como cliente.

1. **Conexión a la red.** `WiFi.begin(ssid, password)` conecta el ESP32 al Wi-Fi. Un bucle espera hasta 10 s a que la conexión se establezca; si no lo consigue, se salta la publicación y sigue el ciclo.
2. **Servidor.** `WiFiServer server(80)` escucha en el puerto 80, el estándar de HTTP. Con `server.begin()` empieza a aceptar conexiones y la IP que imprime por el monitor serie es la que se escribe en el navegador.
3. **Atender al navegador.** Durante una ventana de tiempo (`VENTANA_SERVIDOR`), `atenderCliente()` comprueba si hay un cliente con `server.available()`, lee su petición HTTP y responde con una página HTML.

4. **La página.** Se genera con `client.println(...)`. Recorre las 19 posiciones de la EEPROM y crea una fila de tabla por ángulo; las distancias menores de 20 cm se marcan en rojo como obstáculo.
5. **Cierre.** Al terminar la ventana, `server.stop()` y `WiFi.disconnect(true)` apagan el servidor y la conexión antes de dormir.

Un pequeño script en la página consulta periódicamente al ESP32; si el equipo estaba dormido y vuelve a responder, recarga la página una vez para mostrar el mapa nuevo.

Explicación del código: modo de energía

Se usa **Deep Sleep**, el modo de menor consumo del ESP32: apaga la CPU y WiFi y mantiene activo solo lo mínimo para poder despertar. El ESP32 entra en este modo al final de cada ciclo, y **se despierta con un temporizador**: pasado el tiempo indicado, el chip se reinicia y vuelve a ejecutar `setup()`. En el código el reposo dura 5 segundos para poder probarlo rápido, y en un uso real se ampliaría a varios minutos cambiando un solo valor.

Por qué Deep Sleep. El dispositivo simula un detector colocado, por ejemplo, en un pasillo y alimentado con una batería. No necesita tener el servidor abierto ni la CPU activa de forma continua, porque los trabajadores solo consultarían el mapa de obstáculos de manera puntual. Dormir el chip por completo evita un consumo innecesario y hace el sistema autónomo y eficiente.

Consecuencias de diseño.

- Al despertar el sistema se reinicia por completo, lo que deja pines y servo en un estado limpio en cada escaneo.
- Lo que debe recordarse entre ciclos (el mapa y el último sentido del barrido) se guarda en la EEPROM.
- El WiFi se apaga al dormir, así que la página solo está disponible durante el tiempo en el que el servidor permanece abierto.